F

Every line of my settings.json, annotated

> **Note**
>
> *One real, working* `~/.claude/settings.json` *from a v2-graduate operator (me): every meaningful key followed by the regret or insight that put it there.*

This is one operator's file, dated, sanitized of client paths but otherwise real. The value isn't to copy it. The value is to read the *regrets* and ask whether your file should preempt the same ones. The closing prompt below is the recommended way to use this appendix: paste it into Claude with your own settings.json and let Claude diff against the patterns here.

A note on the JSON syntax: Claude wrote this. You don't need to understand every brace and bracket to read the annotations. Read the annotations first; the JSON is there to anchor the references. Same advice as *Chapter 9*'s hook block.

This block is shown with `//` comments for reading. Strict JSON doesn't allow inline comments: if you copy this verbatim into `settings.json`, strip the comments or save the file as `.jsonc` if your tooling supports it.

The file

```
~/.claude/settings.json
{
  "permissions": {
    "deny": [
      // Floor of safety from Ch 2 — every reader of this book has these three.
      "Bash(rm -rf *)",
      "Bash(rm * ~/work/**)",
      "Bash(sudo *)",

      // Paired upgrades from hiring named specialists.
```

```
      "mcp__gmail__send_*", // added Ch 11 — Iris's
        second lock
      "mcp__notion__create_page(database_id=db_crm_8f3a)", // added Ch 14 — Echo's
        CRM lock
      "mcp__notion__update_page(database_id=db_crm_8f3a)", // added Ch 14
      "mcp__notion__delete_page(database_id=db_crm_8f3a)", // added Ch 14

      // My own scars, accumulated over the years. Yours will look different.
      "Bash(curl * | sh)",
      "Bash(curl * | bash)",
      "Write(~/.ssh/**)",
      "Write(~/Library/**)"
    ],
    "ask": [
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Write(~/work/.claude/settings.local.json)",
      "Write(~/.claude/settings.json)"
    ],
    "allow": [
      "Read(~/work/**)",
      "Edit(~/work/**)",
      "Bash(ls *)",
      "Bash(cat *)",
      "Bash(grep *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(npm test*)"
    ]
  },

  "hooks": {
    "SessionStart": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "~/work/scripts/morning-brief.sh --read-only",
            "timeout": 10
          }
        ]
      }
```

```
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "echo \"[$(date -Iseconds)] $CLAUDE_TOOL_INPUT\" >>
          ~/.claude/logs/audit/bash.log"
          }
        ]
      }
    ],
    "Stop": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "~/work/scripts/long-task-ping.sh",
            "timeout": 5
          }
        ]
      }
    ],
  "SessionEnd": [
    {
      "matcher": "*",
      "hooks": [
        {
          "type": "command",
          "command": "~/work/scripts/end-of-day.sh"
        }
      ]
    }
  ]
},

"apiKeyHelper": "~/work/scripts/keychain-claude-code.sh",

"env": {
  "CLAUDE_DEFAULT_MODEL": "sonnet",
  "CLAUDE_LOG_DIR": "~/.claude/logs"
},
```

```
"mcpServers": {
  "gmail": {
    "command": "npx",
    "args": ["-y", "@anthropic-ai/mcp-gmail@pinned"]
  },
  "calendar": {
    "command": "npx",
    "args": ["-y", "@anthropic-ai/mcp-calendar@pinned"]
  },
  "notion": {
    "command": "npx",
    "args": ["-y", "@notion/mcp-notion@pinned"]
  },
  "slack": {
    "command": "npx",
    "args": ["-y", "@anthropic-ai/mcp-slack@pinned"]
  },
  "buffer": {
    "command": "npx",
    "args": ["-y", "@buffer/mcp-buffer@pinned"] ]
  },
  "personal-data": {
    "command": "node",
    "args": ["~/work/mcp-servers/personal-data/server.js"]
  }
},

"outputStyle": "concise",
"statusLine": {
  "type": "command",
  "command": "~/.claude/statusline.sh"
}
```

The annotations

The numbers refer to logical blocks in the file, not line numbers (those drift between print runs).

permissions

1. **The three baseline deny rules**: `Bash(rm -rf *)`, `Bash(rm * ~/work/**)`, `Bash(sudo *)`. These are the floor of safety from Chapter 2: every reader of the book installs these on day one. The first blocks the catastrophic recursive-delete shell command anywhere on the computer. The second protects the work folder (where the staff lives) from any `rm` regardless of flags. The third blocks elevated-privilege

commands. Three rules, no MCP-level paranoia in the baseline; readers add MCP-scoped rules later as they hire specialists whose drift would matter (Iris in *Chapter 11*, Echo in *Chapter 14*).

2. `Bash(curl * | sh)` **and** `Bash(curl * | bash)` **in deny.** Anything that pipes a network response straight into a shell is a class of action I never want Claude to do unattended. Even from sources I trust. If I want to install something this way, I do it myself.
3. `Write(~/.ssh/**)` **and** `Write(~/Library/**)` **in deny.** Two folders that should never be touched by an agent. The `.ssh` rule is obvious. The `Library` rule is the one that surprised me: I once watched a well-intentioned cleanup script try to "tidy" `Library/Caches` and break the calendar app for an afternoon.
4. **The** `ask` **list: git-force-push and git-reset-hard variants.** The `Bash(rm -rf *)` baseline already blocks recursive deletes outright; the ask list catches the next tier of irreversible operations: force-pushing over a teammate's commits, resetting a branch hard enough to lose work. Both prompt me before executing. The discipline: any operation that erases history, even mine, has to be explicit.
5. `Bash(git push --force *)` **and** `Bash(git reset --hard *)` **in ask.** Both are operations I sometimes want to run and never want an agent to run unattended. The prompt is cheap; the recovery cost when one fires unexpectedly is not.
6. `Write(~/.claude/settings.json)` **and** `Write(~/work/.claude/settings.json)` **in ask.** The file you're reading shouldn't be modified silently. If Claude wants to change my permissions, hooks, or MCP servers, it asks first. Every time.
7. **The** `allow` **list: read access to** `~/work/`. The deliberate trust zone. Everything inside `~/work/` is fair game for reads. Outside, every read prompts. This is the boundary that lets the agents do their job without making my whole filesystem accessible.
8. `Bash(ls *)`, `Bash(cat *)`, `Bash(grep *)`. Read-only shell verbs that I got tired of approving forty times a day. Adding them to allow saved hours over months.
9. `Bash(git status), Bash(git diff *), Bash(git log *)` **in allow.** Read-only git operations. `git status` is the most-run command in any session. Approving it every time was friction with no safety value.

hooks

1. **The** `SessionStart` **hook:** `morning-brief.sh --read-only`. *Chapter 9*'s morning brief reader. The `--read-only` flag is the regret: the original version regenerated the brief on every `claude` invocation, including the seventeen times a day I open a new tab. The brief is now generated by the scheduled job at 6:30 AM only; the SessionStart hook just *reads* the cached file. Cost dropped 90%.
2. **The** `timeout: 10` **on the SessionStart hook.** I once had a session hang for two minutes because the hook script tried to fetch from a slow endpoint. The 10-second cap is the floor below which I'd rather see no brief than wait for one.
3. **The** `PreToolUse` **audit hook on** `Bash`. Every Bash invocation appends a timestamp and the command to `~/.claude/logs/audit/bash.log`. *Chapter 10*'s territory. The regret here is subtle: I added it three months in, after I couldn't reconstruct *which* `rm` command had cleared a directory because nothing was logging Bash invocations. Now I always know. The log is also the input to a quarterly grep when I want to know what shell verbs my agents actually run.

4. **The `Stop` hook:** `long-task-ping.sh`. Fires on every Stop event. The script checks elapsed time and pings Slack only if the task ran longer than 5 minutes. *Chapter 11*'s territory. The regret: the Tuesday I waited 11 minutes for a research run that had finished 8 minutes earlier because I'd Slack'd a colleague and forgotten about it. The hook now catches that case.
5. **The `SessionEnd` hook:** `end-of-day.sh`. Reuben's bookkeeper trigger lives partly here. On `SessionEnd`, the script checks if it's Friday after 4 PM; if yes, it queues Reuben's monthly close. If no, it just files journal notes. The conditional in the script keeps the hook fast on every other invocation. Reuben's run output streams to `~/.claude/logs/jobs/reuben.log` (the per-job log path *Appendix B*'s *log everything* convention recommends: one line per job under `~/.claude/logs/`, named for the hook or job that wrote it).

apiKeyHelper and env

1. `apiKeyHelper: "~/work/scripts/keychain-claude-code.sh"`. The documented mechanism for sourcing an API key from a secret store. The script prints the key on stdout; Claude Code reads it from there. The actual key lives in *macOS Keychain* (the system password store; on Linux substitute Secret Service via `secret-tool`; on Windows substitute Credential Manager): the script is a one-liner like `security find-generic-password -s claude-code -w`. The secret never sits in plain text in a config file. I once committed a settings.json with a literal key. The annotation here is: don't.
2. `CLAUDE_DEFAULT_MODEL: sonnet`. Default for sessions where I haven't specified otherwise. I bump to a heavier model per-agent (in the agent file) when I need it, lighter for triage. The default sits in the middle.
3. **There is no documented per-request token cap env var.** Cost control happens via model selection (cheaper models for cheap tasks) and via `/cost` monitoring: see Appendix B. The earlier draft of this file carried a `CLAUDE_MAX_TOKENS_PER_REQUEST` line; it wasn't a real key and has been removed.
4. `CLAUDE_LOG_DIR: ~/.claude/logs`. Convention setting. Every hook and scheduled job I write knows to use this prefix. Centralizes the log layout so the grep recipes in Appendix B work without per-tool path-hunting.

mcpServers

1. **The five MCPs from Chapter 7: Gmail, Calendar, Notion, Slack, Buffer.** Each pinned at a specific version. The pin annotation: Tier 2 plugin discipline (*Appendix C*). I want the bytes I reviewed to be the bytes that run. Auto-updates from a remote, even from authors I trust, change the runtime under me. The pin lifts every quarter when I deliberately review the new version. (MCP package names and authorship evolve; browse the External integrations section in `/plugin` Discover for current canonical names. The `@pinned` tag is the author's convention for the npm version actually installed at audit time, not a real npm dist-tag.)
2. `personal-data`: **Chapter 16's custom MCP.** Run from a local path. No version pin because there is no version: it's my own code. The annotation is: this is what Chapter 16 built. The path is the only path; the file is the only file.

agents and skills discovery

1. **No** `agents.searchPaths` **or** `skills.searchPaths` **key.** Subagents and skills are auto-discovered from fixed paths: `~/.claude/agents/` and `.claude/agents/`(recursively), and the same for `skills/`. No configurable search-paths key. Privacy-by-separation, in this file's prior draft, was an illusion: the keys don't exist and were silently ignored. If you want a private subset, gitignore individual files inside the canonical folders, or symlink in from a private path.

UI / output preferences

1. `outputStyle: concise`. *Chapter 10* territory. After three weeks of getting prose answers when I wanted lists, I switched. The default verbose mode is fine for someone learning; concise is the operator default.
2. `statusLine`. The real `statusLine` takes an object: `{ "type": "command", "command": "~/.claude/statusline.sh" }`. The script runs and prints the line you want: compose it with whatever shell/python/node command outputs the model + agent + tokens display you want. The token counter is the friction point: I notice when it climbs, which catches cost issues before *Appendix B*'s monthly check would.
3. **Theme is set via the** `/theme` **slash command, not in settings.json.** No top-level `theme` key exists. The earlier draft of this file carried one; it has been removed. (Including this annotation as a reminder that *"the key doesn't exist"* is also fine to know: it tells you where *not* to look.)

channels

1. `channelsEnabled: true`. *Chapter 15* (Pheme) territory. Channels are the Claude Code feature where an MCP server *pushes* inbound events into a running session (Telegram messages, iMessage texts, Discord pings) and Pheme reacts to them while you sleep. On Pro/Max plans the feature is opt-in via this single boolean in managed settings; on Team/Enterprise plans the org admin sets it. Without `channelsEnabled: true`, the `--channels` flag at session launch is a no-op. (Channels are not available on Bedrock/Vertex/Foundry.)
2. **Channel env vars:** `TELEGRAM_BOT_TOKEN, DISCORD_BOT_TOKEN`. Stored in `~/.claude/channels/{telegram,discord}/.env` rather than in `settings.json` itself, because they're plaintext secrets and the canonical secret-store pattern (the API key helper above) doesn't apply: channel plugins read the env file directly at session start. iMessage needs no token: it reads `~/Library/Messages/chat.db` after the user grants macOS Full Disk Access to Terminal. The sender allowlist for each channel is set per-session by `/telegram:access, /imessage:access, /discord:access` and persists in `~/.claude/channels/{channel}/access.json`.

Three things this file deliberately doesn't set

A. No `auto-approve-everything: true`. Don't ever set this. The fact that you'd save two seconds per prompt is not worth the day you find out an agent decided to send 80 emails.

B. No global model override pointing at the most expensive model. Tempting because the most-capable model is genuinely better at hard tasks. But for triage and routine drafts, it's wasted spend. Per-agent overrides handle the cases where the upgrade matters. Defaults stay at sonnet.

C. No `disableTelemetry` or equivalent override. Anthropic's telemetry is documented and minimal. The button to turn it off exists; I don't press it. The trade, supporting the product team's ability to detect issues, is one I'd want my own users to make.

The closing prompt

When you're ready to compare your own file against this one, paste this into a fresh `claude` session:

```
Read ~/.claude/settings.json. Compare it against the patterns described in Appendix F of
"Claude Code for Non-Programmers" — permissions deny/ask/allow shape, hook coverage
(SessionStart, PreToolUse audit, Stop, SessionEnd), apiKeyHelper + env guardrails,
mcpServers pinning, statusLine as a command object. For each pattern from Appendix F that
I'm missing, propose the addition and one sentence on why it's worth having. Don't apply
anything yet — show me the diff first.
```

The output is one page. You read the diff, decide what to take, paste back the green-light. Claude applies. Most readers find that two or three patterns from this file are worth their own. Some find none, and that's a real answer too. The shape of *your* file is the file you regret least, not the file that copies mine.

The schema reference Anthropic publishes is canonical for the full list of keys and what each one does. This appendix is keys *that have a story behind them*. The two are complementary.